

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计

批改人：

实验	并行多源最短路径搜索	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/5/21

1. 实验目的

使用并行技术完成无向图上的多源最短路径搜索。并分析在不同进程数量，数据下该实现的性能。

2. 实验过程和核心代码

由于只需要计算测试文件给定的两点之间的距离。所以使用 Dijkstra 算法进行两点之间的距离计算，同时由于只需要计算两点之间的距离，所以可以进行剪枝，在计算到目标点后就退出。在并行化方面，采用数据并行方式，将任务分配到不同的进程（线程）中独立执行算法，得到结果后聚集到主进程（线程）。

我主要实现了 MPI 版本的代码，作为参照，我也生成了 Pthread、OpenMP 版本的实现方式，实现代码如下：

图表我采用邻接表的组织形式。加载代码如下：

```

void read_graph(const char* filename, AdjList** graph, int* node_count) {

    char line[128];
    fgets(line, sizeof(line), fp); // skip header
    int u, v;
    double w;
    int max_node = 0;

    int cap = 1024;
    *graph = (AdjList*)calloc(cap, sizeof(AdjList));

    while (fgets(line, sizeof(line), fp)) {
        sscanf(line, "%d,%d,%lf", &u, &v, &w);
        if (u >= cap || v >= cap) {
            int new_cap = (u > v ? u : v) + 128;
            *graph = (AdjList*)realloc(*graph, sizeof(AdjList) * new_cap);
            for (int i = cap; i < new_cap; ++i) {
                (*graph)[i].edges = NULL;
                (*graph)[i].edge_count = 0;
                (*graph)[i].edge_capacity = 0;
            }
            cap = new_cap;
        }
        add_edge(*graph, u, v, w);
        add_edge(*graph, v, u, w);
        if (u > max_node) max_node = u;
        if (v > max_node) max_node = v;
    }
    fclose(fp);
    *node_count = max_node + 1;
}

```

Dijkstra 算法代码如下：

```

double dijkstra(AdjList* graph, int n, int src, int tgt) {
    double* dist = (double*)malloc(sizeof(double) * n);
    char* visited = (char*)calloc(n, sizeof(char));
    for (int i = 0; i < n; ++i) dist[i] = INF;
    dist[src] = 0;

    for (int i = 0; i < n; ++i) {
        int u = -1;
        double min_d = INF;
        for (int j = 0; j < n; ++j) {
            if (!visited[j] && dist[j] < min_d) {
                u = j;
                min_d = dist[j];
            }
        }
        if (u == -1 || u == tgt) break;
        visited[u] = 1;

        for (int j = 0; j < graph[u].edge_count; ++j) {
            int v = graph[u].edges[j].node;
            double w = graph[u].edges[j].weight;
            if (!visited[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    double result = dist[tgt];
    free(dist); free(visited);
    return result;
}

```

MPI 版本的代码如下，首先将邻接表等共享数据广播给所有进程，由于邻接表的总体大小难以计算，一次性广播出去比较困难，所以逐步将邻接表进行广播。最后通过 MPI_reduce 将数据收集起来：

```

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    AdjList* graph = NULL;
    int* test_pairs = NULL;
    int node_count = 0, pair_count = 0;

    double start = MPI_Wtime();

    if (rank == 0) {
        read_graph("./updated_flower.csv", &graph, &node_count);
        read_test_pairs("./test_250.csv", &test_pairs, &pair_count);
        printf("Graph loaded\n");
    }

    MPI_Bcast(&node_count, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&pair_count, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank != 0) {
        graph = (AdjList*)calloc(node_count, sizeof(AdjList));
    }
}

```

```

for (int i = 0; i < node_count; ++i) {
    int edge_count = 0;
    if (rank == 0) edge_count = graph[i].edge_count;
    MPI_Bcast(&edge_count, 1, MPI_INT, 0, MPI_COMM_WORLD);
    graph[i].edge_count = edge_count;
    if (edge_count > 0) {
        if (rank != 0) {
            graph[i].edges = (Edge*)malloc(sizeof(Edge) * edge_count);
        }
        MPI_Bcast(graph[i].edges, sizeof(Edge) * edge_count, MPI_BYTE, 0, MPI_COMM_WORLD);
    }
}

if(rank == 0) printf("Graph broadcasted\n");

if (rank != 0) {
    test_pairs = (int*)malloc(sizeof(int) * pair_count * 2);
}
MPI_Bcast(test_pairs, pair_count * 2, MPI_INT, 0, MPI_COMM_WORLD);

if(rank == 0) printf("Test pairs broadcasted\n");

double* local_results = (double*)malloc(sizeof(double) * pair_count);
double* all_results = NULL;
if (rank == 0) {
    all_results = (double*)malloc(sizeof(double) * pair_count);
}

```

```

for (int i = 0; i < pair_count; ++i) {
    if (i % size == rank) {
        int s = test_pairs[2 * i];
        int t = test_pairs[2 * i + 1];
        local_results[i] = dijkstra(graph, node_count, s, t);
    } else {
        local_results[i] = -1;
    }
}

MPI_Reduce(local_results, all_results, pair_count, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0) {
    printf("Results:\n");
    for (int i = 0; i < pair_count; ++i) {
        printf("Test case %d (%d -> %d): %.4f\n", i, test_pairs[2*i], test_pairs[2*i+1], all_
    }
    printf("Task done\n");
    free(all_results);
}
free(local_results);

double end = MPI_Wtime();
if (rank == 0) printf("Total time: %.4f sec\n", end - start);

MPI_Finalize();
return 0;
}

```

Pthread 代码如下:

```

void* worker(void* arg) {
    ThreadArg* ta = (ThreadArg*)arg;
    for (int i = ta->thread_id; i < ta->pair_count; i += ta->num_threads) {
        int s = ta->pairs[2 * i];
        int t = ta->pairs[2 * i + 1];
        results[i] = dijkstra(ta->graph, ta->node_count, s, t);
    }
    return NULL;
}

```

```

int main(int argc, char** argv) {
    AdjList* graph;
    int* test_pairs;
    int node_count, pair_count;
    read_graph("./updated_flower.csv", &graph, &node_count);
    read_test_pairs("./test_250.csv", &test_pairs, &pair_count);

    results = (double*)calloc(pair_count, sizeof(double));
    int num_threads = 4;
    pthread_t threads[num_threads];
    ThreadArg args[num_threads];

    double start = clock() / (double)CLOCKS_PER_SEC;

    for (int i = 0; i < num_threads; ++i) {
        args[i] = (ThreadArg){graph, test_pairs, pair_count, node_count, i, num_threads};
        pthread_create(&threads[i], NULL, worker, &args[i]);
    }
    for (int i = 0; i < num_threads; ++i) {
        pthread_join(threads[i], NULL);
    }

    printf("Results:\n");
    for (int i = 0; i < pair_count; ++i) {
        printf("Test case %d (%d -> %d): %.4f\n", i, test_pairs[2*i], test_pairs[2*i+1], results[i]);
    }

    double end = clock() / (double)CLOCKS_PER_SEC;
    printf("Total time: %.4f sec\n", end - start);

    free(results);
}

```

OpenMp 的代码如下：

```

int main(int argc, char** argv) {
    AdjList* graph;
    int* test_pairs;
    int node_count, pair_count;
    read_graph("./updated_flowier.csv", &graph, &node_count);
    read_test_pairs("./test_250.csv", &test_pairs, &pair_count);

    double* results = (double*)calloc(pair_count, sizeof(double));
    double start = omp_get_wtime();

    #pragma omp parallel for schedule(static)
    for (int i = 0; i < pair_count; ++i) {
        int s = test_pairs[2 * i];
        int t = test_pairs[2 * i + 1];
        results[i] = dijkstra(graph, node_count, s, t);
    }

    printf("Results:\n");
    for (int i = 0; i < pair_count; ++i) {
        printf("Test case %d (%d -> %d): %.4f\n", i, test_pairs[2*i], test_pairs[2*i+1], results[i]);
    }

    double end = omp_get_wtime();
    printf("Total time: %.4f sec\n", end - start);
    free(results);
    return 0;
}

```

使用自定义的测试用例，运行结果展示如下：

```
Test case 221 (110 -> 111): 1.0000
Test case 222 (110 -> 114): 1.0000
Test case 223 (111 -> 112): 1.0000
Test case 224 (111 -> 115): 1.0000
Test case 225 (112 -> 113): 1.0000
Test case 226 (112 -> 116): 1.0000
Test case 227 (113 -> 114): 1.0000
Test case 228 (113 -> 117): 1.0000
Test case 229 (114 -> 115): 1.0000
Test case 230 (114 -> 118): 1.0000
Test case 231 (115 -> 116): 1.0000
Test case 232 (115 -> 119): 3.0000
Test case 233 (116 -> 117): 1.0000
Test case 234 (116 -> 120): 3.0000
Test case 235 (117 -> 118): 1.0000
Test case 236 (117 -> 121): 3.0000
Test case 237 (118 -> 119): 3.0000
Test case 238 (118 -> 122): 3.0000
Test case 239 (119 -> 120): 1.0000
Test case 240 (119 -> 123): 1.0000
Test case 241 (120 -> 121): 1.0000
Test case 242 (120 -> 124): 1.0000
Test case 243 (121 -> 122): 1.0000
Test case 244 (121 -> 125): 1.0000
Test case 245 (122 -> 123): 1.0000
Test case 246 (122 -> 126): 1.0000
Test case 247 (123 -> 124): 1.0000
Test case 248 (123 -> 127): 1.0000
Test case 249 (124 -> 125): 1.0000
Test case 250 (124 -> 128): 1.0000
Test case 251 (125 -> 126): 1.0000
Test case 252 (125 -> 129): 1.0000
Test case 253 (126 -> 127): 1.0000
Test case 254 (126 -> 130): 1.0000
Task done
Total time: 0.0248 sec
```

3. 实验结果

我分别创建了用例数量为 1000、2000、3000 的测试用例文件，以 updated_mopuse.csv 为图表。按照我的硬件条件设置进程（线程）数为 1、2、4、8、16。分别进行运行，收集的运行时间结果如下：

MPI 版本结果如下：

测试用例 数量\进 程数	1	2	4	8	16
1000	0.3289	0.2015	0.1167	0.0834	0.1069
2000	0.7529	0.3979	0.2237	0.1611	0.1400
3000	0.9498	0.4937	0.2839	0.1933	0.1443

使用 updated_flower.csv 文件作为图表，运行相同的测试文件的运行结果如下：

测试用例 数量\进 程数	1	2	4	8	16
1000	0.0459	0.0347	0.0218	0.0317	0.0573
2000	0.2068	0.1117	0.0760	0.0691	0.0802
3000	0.2213	0.1256	0.0798	0.0676	0.0873

从表中可以看到，测试节点的数量显著影响着任务运行时间，但是其余条件相同的情况下，运行时间和节点数量并不是线性的，推测这是因为测试文件的每一个任务运行量是不同的，某些数据运行消耗时间长，有些消耗时间短。导致运行时间与节点数量非线性。

通过表中的结果可以看见，进程数越多运行时间越小。当进程数为 16 时，第二组数据的性能不升反降，推测这是因为进程切换和创建以及数据传递等并行化开销成为了性能瓶颈。

Flower 的平均度数更高，意味着每一个点的邻居比较多，理论来说 Dijkstra 算法的运行时间更长。但是 mouse 的距离都是小数，计算更加复杂，这也侧面说明了平均度数在这个问题中对任务性能的影响并不明显。

对于并行方式的分析，我采用 OpenMP 版本的代码来进行分析，由于这个问题显然是任务不均衡类型的问题，因此理应动态划分的运行效率要比静态划分的运行效率高。在 updated_mouse.csv 作为图表，运行 2000 个测试用例的文件，运行多次的平均结果如下：

静态：

```
Total time: 0.1792 sec
```

动态：

```
Total time: 0.1521 sec
```

可以看到动态模式的性能要高于静态模式，这是因为任务是不均衡的，在静态模式下运行时会有某些任务量小线程处于空闲等待，而动态模式可以实现负载均衡，使得每个线程都不会“偷懒”。从而性能提升。

4. 实验感想

通过这次实验，我实现了使用 MPI、Pthread 以及 OpenMP 的并行化多源最短路径搜索任务。分析了线程数量对并行性能的影响。同时分析比较了数据以及并行方式对并行性能的影响。受益匪浅。